



DAY 1 — SQL FOUNDATION + SELECT



1. What is SQL?

👉 SQL = Structured Query Language

Used to:

- Store data
- Retrieve data
- Update/Delete data

👉 Think like this:

None

Database = Excel file

Table = Sheet

Row = Record

Column = Field



2. What is a Table?

Example: `employees`

id	name	salary	dept
1	Sameer	50000	IT
2	Rahul	60000	HR
3	Anita	70000	IT



3. SELECT — MOST IMPORTANT COMMAND

👉 Used to **read data**

✅ **Syntax:**

SQL

```
SELECT column_name
```

```
FROM table_name;
```

Show more lines

✅ **Example 1: Get all data**

SQL

```
SELECT * FROM employees;
```

Show more lines

👉 * means everything

✅ **Example 2: Specific columns**

SQL

```
SELECT name, salary FROM employees;
```

Show more lines

✅ **Output:**

name	salary
Sameer	50000
Rahul	60000
Anita	70000

🔍 **4. WHERE — Filtering data**

👉 Used to apply conditions

✓ Syntax:

SQL

SELECT *

FROM employees

WHERE condition;

Show more lines

✓ Example:

SQL

SELECT *

FROM employees

WHERE salary > 60000;

Show more lines

✓ Output:

id	name	salary	dept
3	Anita	70000	IT

5. Operators

Operator	Meaning
=	Equal
>	Greater
<	Less
>=	Greater or equal
<=	Less or equal
!=	Not equal

6. AND / OR

AND (both conditions must be true)

SQL
SELECT *
FROM employees
WHERE dept = 'IT' AND salary > 60000;
[Show more lines](#)

OR (any one condition true)

SQL
SELECT *
FROM employees
WHERE dept = 'IT' OR salary > 60000;
[Show more lines](#)

YOUR PRACTICE (VERY IMPORTANT)

Use this table mentally or in any SQL tool:

None
`employees`
`(id, name, salary, dept)`

Task 1

 Get all employees

Task 2

 Get only names

Task 3

 Get employees with salary > 55000

Task 4

 Get employees from IT department

Task 5 (Interview Level)

 Get employees from IT dept with salary > 50000

Answers (Try first, then check)

Click to expand answers

SQL

-- Task 1

```
SELECT * FROM employees;
```

-- Task 2

```
SELECT name FROM employees;
```

-- Task 3

```
SELECT * FROM employees WHERE salary > 55000;
```

-- Task 4

```
SELECT * FROM employees WHERE dept = 'IT';
```

-- Task 5

```
SELECT * FROM employees
```

```
WHERE dept = 'IT' AND salary > 50000;
```

Show more lines



MEMORY TRICK (VERY IMPORTANT)

Think in this order:

None

SELECT → WHAT to show

FROM → WHICH table

WHERE → FILTER condition



WHAT YOU LEARNED TODAY

- ✓ SELECT
- ✓ WHERE
- ✓ Basic filtering
- ✓ AND / OR



DAY 2 — SORTING + UNIQUE + PATTERN SEARCH



Today's Goal

You'll learn how to: ✓ Sort data

- ✓ Get unique values
 - ✓ Search like Google (pattern matching)
 - ✓ Use range and list filtering
-



1. ORDER BY (Sorting Data)

👉 Used to **sort results**

✅ **Syntax:**

SQL

```
SELECT column_name
```

```
FROM table_name
```

```
ORDER BY column_name ASC|DESC;
```

Show more lines

✅ **Example 1: Ascending (default)**

SQL

```
SELECT name, salary
```

```
FROM employees
```

```
ORDER BY salary;
```

Show more lines

👉 Lowest → Highest

✅ **Example 2: Descending**

SQL

```
SELECT name, salary
```

```
FROM employees
```

```
ORDER BY salary DESC;
```

Show more lines

👉 Highest → Lowest

✅ **Example 3: Multiple columns**

SQL

```
SELECT *
```

```
FROM employees
ORDER BY dept ASC, salary DESC;
Show more lines
```

👉 First sort by dept, then salary inside dept



MEMORY TIP

None

ORDER BY → arrange data like Excel sorting

2. DISTINCT (Remove Duplicates)

👉 Used to get **unique values**

✅ Example:

```
SQL
SELECT DISTINCT dept
FROM employees;
Show more lines
```

✅ Output:

None

IT
HR

👉 Even if multiple employees in IT → shows once

3. LIKE (Pattern Search)

👉 Used for **searching text**

Wildcards

Symbol	Meaning
%	Any characters
_	Single character

Example 1: Starts with 'S'

SQL

```
SELECT *
```

```
FROM employees
```

```
WHERE name LIKE 'S%';
```

Show more lines

👉 Sameer, Suresh, etc.

Example 2: Ends with 'I'

SQL

```
SELECT *
```

```
FROM employees
```

```
WHERE name LIKE '%I';
```

Show more lines

👉 Rahul, etc.

Example 3: Contains 'it'

SQL

```
SELECT *
```

```
FROM employees
```

```
WHERE name LIKE '%it%';
```

Show more lines

✓ Example 4: Exact length match

SQL

```
SELECT *
```

```
FROM employees
```

```
WHERE name LIKE 'R_____';
```

Show more lines

👉 5-letter names starting with R



4. BETWEEN (Range Filter)

👉 Used for **range values**

✓ Example:

SQL

```
SELECT *
```

```
FROM employees
```

```
WHERE salary BETWEEN 40000 AND 70000;
```

Show more lines

👉 Includes both 40000 and 70000



5. IN (Multiple Values)

👉 Used instead of multiple OR

✅ Example:

```
SQL
SELECT *
FROM employees
WHERE dept IN ('IT', 'HR');
```

Show more lines

👉 Same as:

```
SQL
WHERE dept = 'IT' OR dept = 'HR'
```

Show more lines

YOUR PRACTICE (VERY IMPORTANT)

Use same table:

None
`employees (id, name, salary, dept)`

Task 1

👉 Show all employees sorted by salary (high to low)

Task 2

👉 Show unique departments

Task 3

👉 Find employees whose name starts with 'A'

Task 4

👉 Find employees with salary between 50000 and 70000

Task 5

👉 Get employees from IT and HR departments only

Task 6 (Interview type)

👉 Find employees whose name contains 'a'

Answers (Try first!)

Click to expand answers

SQL

-- Task 1

```
SELECT * FROM employees ORDER BY salary DESC;
```

-- Task 2

```
SELECT DISTINCT dept FROM employees;
```

-- Task 3

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

-- Task 4

```
SELECT * FROM employees WHERE salary BETWEEN 50000 AND 70000;
```

-- Task 5

```
SELECT * FROM employees WHERE dept IN ('IT', 'HR');
```

-- Task 6

```
SELECT * FROM employees WHERE name LIKE '%a%';
```

Show more lines



DAY 2 MEMORY SUMMARY

None

SELECT → choose data

FROM → table

WHERE → filter

ORDER BY → sort

DISTINCT → unique

LIKE → search

BETWEEN → range

IN → list



DAY 3 — AGGREGATION + GROUP BY + HAVING



Today's Goal

You'll learn how to: ☒ Calculate totals, averages

☒ Group data (like reports)

☒ Filter grouped data (very important for interviews)



1. AGGREGATE FUNCTIONS

👉 Used to perform calculations on rows

✅ Functions You MUST Know

Function	Meaning
COUNT()	Number of rows
SUM()	Total
AVG()	Average
MIN()	Minimum
MAX()	Maximum

✅ Example Table

```
None
employees
(id, name, salary, dept)
```

✅ Example 1: Count employees

SQL

```
SELECT COUNT(*) FROM employees;
```

Show more lines

✅ Example 2: Total salary

SQL

```
SELECT SUM(salary) FROM employees;
```

Show more lines

✓ Example 3: Average salary

SQL

```
SELECT AVG(salary) FROM employees;
```

Show more lines

✓ Example 4: Min / Max

SQL

```
SELECT MIN(salary), MAX(salary) FROM employees;
```

Show more lines

🟡 2. GROUP BY (🔥 VERY IMPORTANT)

👉 Used to **group rows** and **apply aggregation**

🧠 Think Like This:

None

GROUP BY = Make categories → then calculate

✓ Example: Avg salary per department

SQL

```
SELECT dept, AVG(salary)
```

```
FROM employees
```

```
GROUP BY dept;
```

Show more lines

✓ Output:

dept	avg_salary
IT	60000
HR	55000

✓ **Example: Count employees per dept**

SQL

```
SELECT dept, COUNT(*)
```

```
FROM employees
```

```
GROUP BY dept;
```

Show more lines

! IMPORTANT RULE

👉 If you use GROUP BY:

- Every column must be either:
 - In GROUP BY
 - OR inside a function (AVG, COUNT, etc.)

✓ Correct:

SQL

```
SELECT dept, AVG(salary)
```

```
FROM employees
```

```
GROUP BY dept;
```

Show more lines

✗ Wrong:

SQL

```
SELECT name, dept, AVG(salary)
```

```
FROM employees
```

```
GROUP BY dept;
```


Show more lines

3. HAVING (INTERVIEW FAVORITE)

👉 Used to filter **GROUPED** data

Difference:

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY

✅ Example:

👉 Departments with avg salary > 55000

SQL

```
SELECT dept, AVG(salary)
FROM employees
GROUP BY dept
HAVING AVG(salary) > 55000;
```

Show more lines

COMBINED EXAMPLE (VERY IMPORTANT)

SQL

```
SELECT dept, COUNT(*) AS total_emp, AVG(salary) AS avg_sal
```

```
FROM employees
WHERE salary > 40000
GROUP BY dept
HAVING COUNT(*) > 2;
Show more lines
```

👉 Flow:

1. WHERE → filter rows
 2. GROUP BY → group
 3. HAVING → filter groups
-

YOUR PRACTICE

Task 1

👉 Count total employees

Task 2

👉 Find average salary

Task 3

👉 Find total salary per department

Task 4

👉 Count employees in each department

🔥 Task 5

👉 Show departments where avg salary > 60000

🔥 Task 6 (Interview Level)

👉 Show departments with more than 2 employees

✅ Answers (Try first!)

Click to expand answers

SQL

-- Task 1

```
SELECT COUNT(*) FROM employees;
```

-- Task 2

```
SELECT AVG(salary) FROM employees;
```

-- Task 3

```
SELECT dept, SUM(salary)
FROM employees
GROUP BY dept;
```

-- Task 4

```
SELECT dept, COUNT(*)
FROM employees
GROUP BY dept;
```

-- Task 5

```
SELECT dept, AVG(salary)
FROM employees
GROUP BY dept
HAVING AVG(salary) > 60000;
```

-- Task 6

```
SELECT dept, COUNT(*)
FROM employees
GROUP BY dept
HAVING COUNT(*) > 2;
Show more lines
```



DAY 3 MEMORY TRICK (IMPORTANT)

None

```
SELECT → WHAT
FROM → TABLE
WHERE → filter rows
GROUP BY → group data
HAVING → filter groups
```



WHAT YOU LEARNED

- ✓ COUNT, SUM, AVG, MIN, MAX
- ✓ GROUP BY (MOST IMPORTANT 🔥)
- ✓ HAVING (interview killer 100)

DAY 4 — JOINS (MUST MASTER)

Today's Goal

You'll learn:  How tables are connected

 How to combine data

 INNER, LEFT, RIGHT JOIN

1. WHY JOINS?

 In real systems, data is split across tables

Example Tables

♦ employees

id	name	dept_id
1	Sameer	101
2	Rahul	102
3	Anita	101

♦ departments

id	dept_name
101	IT
102	HR

👉 Now question: “**Show employee name + department name**”

! Problem:

- dept_name is in another table
- ✅ Solution → JOIN

2. INNER JOIN (🔥 MOST USED)

👉 Returns **only matching records**

✅ Syntax:

SQL

SELECT columns

FROM table1 t1

INNER JOIN table2 t2

ON t1.column = t2.column;

Show more lines

✅ Example:

SQL

SELECT e.name, d.dept_name

FROM employees e

INNER JOIN departments d

ON e.dept_id = d.id;

Show more lines

✅ Output:

name	dept_name
Sameer	IT
Rahul	HR
Anita	IT

Memory Trick:

None

INNER JOIN = match only

3. LEFT JOIN

👉 Returns:

- All records from LEFT table
 - Matching from right (if exists)
 - Otherwise NULL
-

Example:

Add one more employee:

| 4 | Rohan | 999 |

(no matching dept)

Query:

SQL

```
SELECT e.name, d.dept_name
```

```
FROM employees e
```

```
LEFT JOIN departments d
```

```
ON e.dept_id = d.id;
```

Show more lines

✓ Output:

name	dept_name
Sameer	IT
Rahul	HR
Anita	IT
Rohan	NULL



Memory Trick:

None

LEFT JOIN = keep all LEFT data



4. RIGHT JOIN

👉 Opposite of LEFT JOIN

- All data from RIGHT table
 - Match from LEFT
-

✓ Example:

SQL

```
SELECT e.name, d.dept_name
```

```
FROM employees e
```

```
RIGHT JOIN departments d
```

```
ON e.dept_id = d.id;
```

Show more lines

👉 If dept exists but no employee → still shown



5. VISUAL UNDERSTANDING

None

INNER JOIN → overlap only

LEFT JOIN → full left + overlap

RIGHT JOIN → full right + overlap



IMPORTANT INTERVIEW POINTS

✓ Always use aliases:

SQL

```
employees e
```

```
departments d
```

Show more lines

✓ JOIN condition:

SQL

```
ON e.dept_id = d.id
```

Show more lines

✓ Without ON → WRONG ✗



COMMON MISTAKE

✗ Wrong:

SQL

SELECT *

FROM employees, departments;

Show more lines

👉 This creates **Cartesian product (wrong)**



YOUR PRACTICE



Task 1

👉 Show employee name and department name



Task 2

👉 Show all employees even if no department



Task 3

👉 Show only employees with valid departments

Task 4 (Interview Level)

 Find employees who don't have department

Task 5 (Important)

 Count employees in each department

Answers

Click to expand answers

SQL

-- Task 1

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.id;
```

-- Task 2

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.id;
```

-- Task 3

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.id;
```

-- Task 4

```
SELECT e.name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.id
WHERE d.id IS NULL;
```

-- Task 5

```
SELECT d.dept_name, COUNT(e.id)
FROM departments d
LEFT JOIN employees e
ON e.dept_id = d.id
GROUP BY d.dept_name;
Show more lines
```



DAY 4 MEMORY SUMMARY

None

```
JOIN = combine tables
INNER → matching only
LEFT → all left + match
RIGHT → all right + match
```



WHAT YOU LEARNED

- ✓ Why joins are needed
- ✓ INNER JOIN (MOST IMPORTANT 🔥)
- ✓ LEFT JOIN (real-world use 🔥🔥)
- ✓ RIGHT JOIN basics
- ✓ NULL handling

DAY 5 — SUBQUERIES (NESTED QUERIES)

Today's Goal

You'll learn: ☒ Query inside another query

☒ Solve complex problems

☒ EXISTS / NOT EXISTS (interview )

1. WHAT IS A SUBQUERY?

 A query inside another query

 Think like this:

None

Outer Query → uses result of Inner Query

☒ Basic Example

 Find employees earning more than average salary

SQL

```
SELECT *
```

```
FROM employees
```

```
WHERE salary > (
```

```
  SELECT AVG(salary)
```

```
  FROM employees
```

```
);
```

Show more lines

✓ Inner Query:

None

`SELECT AVG(salary) → returns one value`

✓ Outer Query:

None

`Compare salary > that value`

2. TYPES OF SUBQUERIES

✓ 1. SCALAR SUBQUERY (returns single value)

👉 Example:

SQL

SELECT name

FROM employees

WHERE salary = (

SELECT MAX(salary) FROM employees

);

Show more lines

👉 Finds highest salary employee

✓ 2. MULTIPLE VALUE SUBQUERY

👉 Returns multiple rows → use IN

✅ **Example:**

```
SQL
SELECT *
FROM employees
WHERE dept_id IN (
  SELECT id
  FROM departments
  WHERE dept_name = 'IT'
);
```

Show more lines

👉 Step-by-step:

1. Find dept_id for IT
 2. Get employees in those departments
-

🔴 3. CORRELATED SUBQUERY (🔥 interview level)

👉 Inner query depends on outer query

✅ **Example:**

👉 Employees earning more than avg salary of their department

```
SQL
SELECT e1.name, e1.salary, e1.dept_id
FROM employees e1
WHERE salary > (
  SELECT AVG(e2.salary)
```

```
FROM employees e2
WHERE e2.dept_id = e1.dept_id
);
```

Show more lines

🧠 Explanation:

- For each employee → calculate dept avg
 - Compare individually
-

🔥 4. EXISTS (VERY IMPORTANT)

👉 Checks if **data exists** or not

✅ Example:

```
SQL
SELECT name
FROM employees e
WHERE EXISTS (
  SELECT 1
  FROM departments d
  WHERE e.dept_id = d.id
);
```

Show more lines

👉 Returns employees **only if department exists**

🔥 5. NOT EXISTS

👉 Opposite of EXISTS

✅ Example:

```
SQL
SELECT name
FROM employees e
WHERE NOT EXISTS (
  SELECT 1
  FROM departments d
  WHERE e.dept_id = d.id
);
```

Show more lines

👉 Finds employees **without valid department**



IN vs EXISTS (INTERVIEW QUESTION)

IN	EXISTS
Compares values	Checks existence
Good for small data	Better for large data



YOUR PRACTICE

🔥 Task 1

👉 Find employee with highest salary

🔥 Task 2

👉 Find employees earning more than average salary

🔥 Task 3

👉 Find employees working in IT department (using subquery)

🔥 Task 4 (Interview Level)

👉 Find employees earning more than dept average

🔥 Task 5

👉 Find employees who don't have department (using NOT EXISTS)

✅ Answers

Click to expand answers

SQL

-- Task 1

SELECT name

FROM employees

WHERE salary = (SELECT MAX(salary) FROM employees);

```
-- Task 2
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

```
-- Task 3
SELECT *
FROM employees
WHERE dept_id IN (
SELECT id FROM departments WHERE dept_name = 'IT'
);
```

```
-- Task 4
SELECT e1.name
FROM employees e1
WHERE salary > (
SELECT AVG(e2.salary)
FROM employees e2
WHERE e2.dept_id = e1.dept_id
);
```

```
-- Task 5
SELECT name
FROM employees e
WHERE NOT EXISTS (
SELECT 1
FROM departments d
WHERE e.dept_id = d.id
);
```

Show more lines



DAY 5 MEMORY SUMMARY

None

Subquery = query inside query

Scalar → one value

IN → multiple values

Correlated → depends on outer query

EXISTS → checks existence



WHAT YOU LEARNED

- ✓ Subqueries (core concept 🔥)
- ✓ Correlated queries (advanced 🔥🔥)
- ✓ EXISTS / NOT EXISTS
- ✓ Real interview scenarios



DAY 6 — WINDOW FUNCTIONS



Today's Goal

You'll learn: ✓ Ranking (top performers)

- ✓ Row numbering
 - ✓ Partitioning (group-wise logic)
 - ✓ Running calculations
-



1. WHAT IS WINDOW FUNCTION?

👉 Performs calculation **without grouping rows**



KEY DIFFERENCE

GROUP BY	WINDOW FUNCTION
Combines rows	Keeps all rows
One result per group	Result for each row

✓ Example

👉 With GROUP BY:

```
SQL
SELECT dept, AVG(salary)
FROM employees
GROUP BY dept;
Show more lines
```

👉 With WINDOW FUNCTION:

```
SQL
SELECT name, dept, salary,
AVG(salary) OVER (PARTITION BY dept) AS avg_sal
FROM employees;
Show more lines
```

✓ Output keeps all employees + shows dept average

1 2 3 4 2. ROW_NUMBER() (🔥 VERY IMPORTANT)

👉 Gives unique number to each row

✓ Example:

SQL

```
SELECT name, salary,  
ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM employees;  
Show more lines
```

✓ Output:

None

```
1 → highest salary  
2 → second highest
```



3. RANK() vs DENSE_RANK()

✓ RANK()

👉 Skips numbers if tie

SQL

```
SELECT name, salary,  
RANK() OVER (ORDER BY salary DESC) AS rank  
FROM employees;  
Show more lines
```

✓ DENSE_RANK()

👉 No gaps

SQL

```
SELECT name, salary,
```

DENSE_RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;
[Show more lines](#)

Example:

salary	RANK	DENSE_RANK
100	1	1
100	1	1
90	3	2

4. PARTITION BY (VERY IMPORTANT)

 Works like GROUP BY but keeps rows

Example:

 Rank employees inside each department

SQL

```
SELECT name, dept, salary,  
RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rank  
FROM employees;  
Show more lines
```

 Output:

None

IT → ranked separately

HR → ranked separately

5. RUNNING TOTAL (INTERVIEW FAVORITE)

Example:

SQL

```
SELECT name, salary,  
SUM(salary) OVER (ORDER BY id) AS running_total
```

```
FROM employees;
```

Show more lines

 Each row shows cumulative total

6. REAL INTERVIEW QUESTIONS

Top 1 employee per department

SQL

```
SELECT *  
FROM (  
SELECT name, dept, salary,  
ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn  
FROM employees
```



```
) t  
WHERE rn = 1;  
Show more lines
```

Second highest salary

```
SQL  
SELECT *  
FROM (  
  SELECT name, salary,  
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
  FROM employees  
) t  
WHERE rnk = 2;  
Show more lines
```

YOUR PRACTICE

Task 1

👉 Assign row number based on salary (highest first)

Task 2

👉 Rank employees based on salary

Task 3

👉 Rank employees within each department

Task 4 (Important)

👉 Find top 2 employees in each department

Task 5 (Interview Level)

👉 Find second highest salary

Answers

Click to expand answers

SQL

-- Task 1

```
SELECT name, salary,  
ROW_NUMBER() OVER (ORDER BY salary DESC)  
FROM employees;
```

-- Task 2

```
SELECT name, salary,  
RANK() OVER (ORDER BY salary DESC)  
FROM employees;
```

-- Task 3

```
SELECT name, dept, salary,  
RANK() OVER (PARTITION BY dept ORDER BY salary DESC)  
FROM employees;
```

-- Task 4

```
SELECT *  
FROM (  
SELECT name, dept, salary,
```

```
RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
FROM employees
)t
WHERE rnk <= 2;
```

```
-- Task 5
SELECT *
FROM (
SELECT name, salary,
DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees
)t
WHERE rnk = 2;
Show more lines
```



DAY 6 MEMORY SUMMARY

None

OVER() → window function engine
ORDER BY → sorting inside window
PARTITION BY → grouping without merging
ROW_NUMBER → unique ranking
RANK → skip numbers
DENSE_RANK → no gaps



WHAT YOU LEARNED

- ✓ Window functions (ADVANCED 🔥)
- ✓ Ranking problems
- ✓ Partition-based calculations
- ✓ Real interview queries

DAY 7 — CTE + VIEWS + INDEX + OPTIMIZATION

Today's Goal

You'll learn:  CTE (clean + powerful queries)

 Views (virtual tables)

 Index (performance boost ⚡)

 Query optimization (interview + real projects)

1. CTE (Common Table Expression)

👉 Temporary result set inside query

👉 Makes complex queries **clean & readable**

Syntax:

SQL

```
WITH cte_name AS (
```

```
SELECT ...
```

```
)
```

```
SELECT * FROM cte_name;
```

Show more lines

Example:

👉 Employees earning above average

SQL

```
WITH avg_salary AS (  
  SELECT AVG(salary) AS avg_sal  
  FROM employees  
)  
SELECT name, salary  
FROM employees, avg_salary  
WHERE salary > avg_sal;  
Show more lines
```

Why CTE is Important:

- Cleaner than subqueries
 - Reusable
 - Used in **complex interview queries**
-

2. CTE vs Subquery

Subquery	CTE
Hard to read	Easy to read 
Nested	Structured 
Limited reuse	Reusable 

3. VIEWS (Virtual Table)

👉 Stored query → behaves like a table

 **Syntax:**

SQL

```
CREATE VIEW high_salary_emp AS
```

```
SELECT name, salary
```

```
FROM employees
```

```
WHERE salary > 60000;
```

Show more lines

✓ **Use:**

SQL

```
SELECT * FROM high_salary_emp;
```

Show more lines

🔥 Why Views?

- Security (hide columns)
 - Reusability
 - Simplify complex queries
-

🔵 4. INDEX (Performance Booster ⚡)

👉 Makes queries **faster**

🧠 Think like this:

None

Without index → scan entire book

With index → jump to page directly ✓

✓ **Syntax:**

SQL

```
CREATE INDEX idx_salary  
ON employees(salary);
```

Show more lines

When to Use Index:

- ✓ Frequently searched columns
 - ✓ JOIN columns
 - ✓ WHERE conditions
-

When NOT to Use:

- ✗ Small tables
 - ✗ Columns with frequent updates
-

5. QUERY OPTIMIZATION (VERY IMPORTANT)

✓ Rule 1: Avoid **SELECT ***

SQL

--  Bad

```
SELECT * FROM employees;
```

--  Good

```
SELECT name, salary FROM employees;
```

Show more lines

✓ Rule 2: Use Index on filters

SQL

```
WHERE salary > 50000
```

Show more lines

✓ Rule 3: Avoid unnecessary joins

✓ Rule 4: Use LIMIT

SQL

```
SELECT * FROM employees LIMIT 10;
```

Show more lines

✓ Rule 5: Use proper JOIN instead of subquery (sometimes)

6. REAL INTERVIEW PROBLEMS

✓ Top 3 salaries using CTE

SQL

```
WITH ranked AS (  
SELECT name, salary,  
DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
FROM employees  
)
```



```
SELECT *  
FROM ranked  
WHERE rnk <= 3;  
Show more lines
```

Remove duplicates

```
SQL  
WITH duplicates AS (  
  SELECT *,  
  ROW_NUMBER() OVER (PARTITION BY name ORDER BY id) AS rn  
  FROM employees  
)  
DELETE FROM duplicates  
WHERE rn > 1;  
Show more lines
```

Salary higher than department average (CTE)

```
SQL  
WITH dept_avg AS (  
  SELECT dept, AVG(salary) AS avg_sal  
  FROM employees  
  GROUP BY dept  
)  
SELECT e.name, e.salary  
FROM employees e  
JOIN dept_avg d  
ON e.dept = d.dept  
WHERE e.salary > d.avg_sal;  
Show more lines
```



YOUR FINAL PRACTICE

Task 1

👉 Write a CTE to find avg salary and filter employees

Task 2

👉 Create a view for IT employees

Task 3

👉 Add index on dept_id

Task 4 (Interview Level)

👉 Get top 3 salaries using CTE

Task 5 (Real-world)

👉 Optimize a query fetching employees by salary

Answers

Click to expand answers

SQL

-- Task 1

```
WITH avg_sal AS (  
  SELECT AVG(salary) AS avg_s FROM employees  
)  
SELECT *  
FROM employees, avg_sal  
WHERE salary > avg_s;
```

-- Task 2

```
CREATE VIEW it_employees AS  
SELECT *  
FROM employees  
WHERE dept = 'IT';
```

-- Task 3

```
CREATE INDEX idx_dept  
ON employees(dept_id);
```

-- Task 4

```
WITH ranked AS (  
  SELECT name, salary,  
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
  FROM employees  
)  
SELECT * FROM ranked WHERE rnk <= 3;
```

-- Task 5

-- Use index + avoid select *

```
SELECT name, salary  
FROM employees  
WHERE salary > 50000;  
Show more lines
```



DAY 7 MEMORY SUMMARY

None

CTE → cleaner queries

VIEW → virtual table

INDEX → faster queries

OPTIMIZATION → improve performance



CONGRATS SAMEER — YOU COMPLETED CORE SQL

You now know: ☒ Basics

☒ Joins

☒ Subqueries

☒ Window functions

☒ CTE + optimization

👉 This is **INTERVIEW-READY LEVEL** 🔥